Administración de Sistemas Operativos

UD6. Scripting

David Ubide Alaiz



1.¿Qué es un script en programación?

Conjunto de instrucciones o comandos escritos en un lenguaje de programación, diseñado para automatizar tareas o añadir funcionalidad dinámica a un sistema o aplicación

https://www.freecodecamp.org/espanol/news/tutorial-de-programacion-de-bash-script-de-shell-de-linux-y-linea-de-comandos-para-principiantes/

2.Scripting Básico en Linux

comando [OPCIONES] argumentos como por ejemplo:

- **date**: Muestra la fecha actual
- **pwd**: Muestra la carpeta de trabajo presente.
- **ls:** Lista los contenidos de la carpeta actual.
- **echo:** Imprime una cadena de texto, o el valor de una variable a la terminal.
- **man**: Da la página de referencia para la documentación

https://www.freecodecamp.org/espanol/news/tutorial-de-programacion-de-bash-script-de-shell-de-linux-y-linea-de-comandos-para-principiantes/

2.Scripting Básico en Linux

Lista de comandos más útiles.

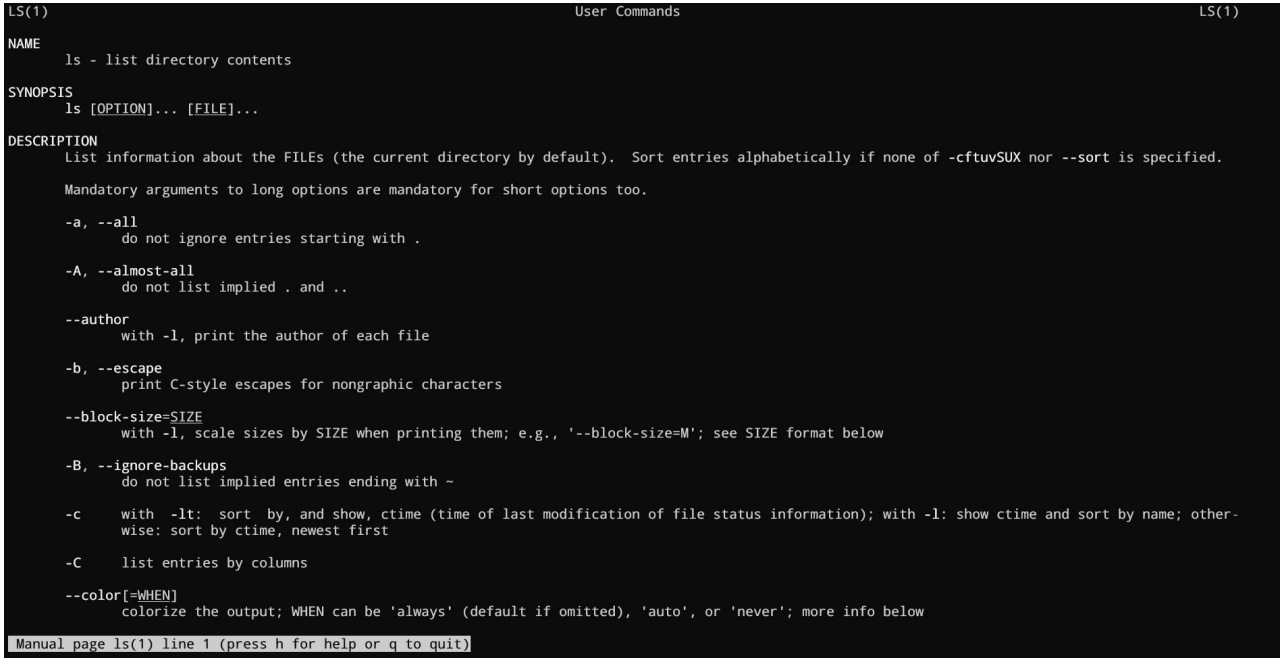
1. **cd**: Cambia el directorio a un lugar distinto.
2. **ls**: Lista los contenidos del directorio actual.
3. **mkdir**: Crea un nuevo directorio.
4. **touch**: Crea un nuevo archivo.
5. **rm**: Elimina un archivo o un directorio.
6. **cp**: Copia un archivo o un directorio.
7. **mv**: Mueve o renombra un archivo o un directorio.

2.Scripting Básico en Linux

Lista de comandos más útiles.

1. **echo**: Imprime texto a la terminal.
2. **cat**: Concatena e imprime los contenidos de un archivo.
3. **grep**: Busca un patrón en un archivo.
4. **chmod**: Cambia los permisos de un archivo o de un directorio.
5. **sudo**: Ejecutar un comando con privilegios administrativos.
6. **df**: Mostrar la cantidad de espacio de disco disponible.
7. **history**: Mostrar una lista de comandos ejecutados previamente.
8. **ps**: Mostrar información sobre procesos en ejecución.

2. Scripting Básico en Linux



3.Primeros pasos

Por convención de nomenclatura, los scripts de bash terminan con **.sh**. Sin embargo, los scripts de bash pueden ejecutarse perfectamente sin la extensión **sh**.
Los scripts de Bash comienzan con un **shebang**. **Shebang** es una combinación de **bash #** y **bang !** seguido del path de la shell de bash. Esta es la primer línea del script. Shebang le dice a la shell que lo ejecute por medio de la shell de bash. Shebang es simplemente un path absoluto al intérprete de bash.

#!/bin/bash

Puedes encontrar tu path de la shell de bash (el cual podría variar del de arriba) usando el comando: **which bash**

3.1 Declaración de variables

1- Asignar el valor directamente:

pais=Pakistan

2. Asignar el valor basado en la salida obtenida de un programa o de un comando, usando la substitución de comando. Ten en cuenta que $ se requiere para acceder a un valor de una variable existente.

mismo_pais=$pais

¿mismo_pais=echo “España”?

¡¡ Cuidado con los espacios en los scripts de Bash !!

3.1 Declaración de variables

3- Capturando la salida de comandos

```
pais=$(echo "España")
echo "$pais"
```



Ejemplo

zaira@Zaira:~$ pais=Pakistan
zaira@Zaira:~$ echo $pais
¿?
zaira@Zaira:~$ nuevo_pais=$pais
zaira@Zaira:~$ echo $nuevo_pais
¿?

Ejemplo

```
zaira@Zaira:~$ pais=Pakistan
zaira@Zaira:~$ echo $pais
Pakistan
zaira@Zaira:~$ nuevo_pais=$pais
zaira@Zaira:~$ echo $nuevo_pais
Pakistan
```

3.1 Declaración de variables

4- Capturando de la entrada del script;

- $1 , $2
- $@ coge las variables separadas por espacios
- $# coge el número de variables de bash

```
#!/bin/bash
echo "Cantidad de parámetros: $#"
for a in "$@"; do
  echo "$a"
done
echo "Cantidad de argumentos utilizados: $#"
```

3.2 Permisos dentro de linux

El comando chmod (change mode) sirve para gestionar los permisos de los archivos o directorios del sistema. Cuando usamos este comando debemos tener presentes los tres niveles de gestión de permisos que existen:

1. Lectura (r)
2. Escritura (w)
3. Ejecución (x)

Además, tener que tener en cuenta a los **tres tipos de usuarios** a los que podemos asignar permisos de manera independiente:

1. Propietario (Owner): El dueño del archivo (o directorio)
2. Grupo (Group): El grupo al que pertenece
3. Otros (Others): Todos los usuarios en general

https://www.arsys.es/blog/comandos-chmod-chown

3.2 Permisos dentro de linux

El comando chmod (change mode) sirve para gestionar los permisos de los archivos o directorios del sistema. Cuando usamos este comando debemos

La **tabla de transformación de octal a binario** sería la siguiente:

Número	Binario	Lectura (r)	Escritura (w)	Ejecución (x)
0	000	No	No	No
1	001	No	No	Sí
2	010	No	Sí	No
3	011	No	Sí	Sí
4	100	Sí	No	No
5	101	Sí	No	Sí
6	110	Sí	Sí	No
7	111	Sí	Sí	Sí

https://www.arsys.es/blog/comandos-chmod-chown

3.2 Permisos dentro de linux

Así, 0 significa que no hay ningún permiso y 7 que los tiene todos.

1. 000 significa que nadie tiene permiso para hacer nada con el fichero.
2. 777 significa que todo el mundo tiene permisos para hacer cualquier cosa.
3. 700 significa que existen permisos totales para el dueño del archivo, pero no para el resto de los usuarios.

chmod 764 archivo.txt

chmod -R 755 mi_carpeta

3.2 Permisos dentro de linux

Modo simbólico:

1. **chmod u+rwx archivo.txt** # Otorga permisos de lectura, escritura y ejecución al usuario chmod g+rw archivo.txt # Permite lectura y escritura al grupo chmod o-r archivo.txt # Quita el permiso de lectura a otros

Modo numérico:

2. **chmod 755 script.sh** # Usuario: lectura, escritura, ejecución; Grupo y otros: lectura, ejecución chmod 644 documento.txt # Usuario: lectura y escritura; Grupo y otros: solo lectura

3.2 Permisos dentro de linux

Para evitar vulnerabilidades en un servidor web, sigue estas recomendaciones:

1. **Archivos públicos:**
   chmod 644 index.html
   # Todos pueden leer, pero solo el propietario puede escribir

2. **Directorios:**
   chmod 755 /var/www/html
   # Permite lectura y ejecución para todos, pero solo el dueño puede modificar

3. **Archivos sensibles (configuraciones, credenciales, etc.):**
   chmod 600 config.php **# Solo el propietario puede leer y escribir**

3.2 Permisos dentro de linux

El comando chown (change owner) sirve para indicar quién es el dueño y el grupo de un archivo. Se puede especificar el usuario (en este caso usuariox) y el grupo (grupoy), separados por el carácter.

chown usuariox:grupoy archivo.txt

chown -R www-data:www-data /var/www/html

Esto se puede observar al realizar el comando ls -l sobre un archivo

ls -l archivo.txt
-rw-r--r-- 1 usuario grupo 1234 Feb 1 12:34 archivo.txt

3.2 Permisos dentro de linux

El comando chown (change owner) sirve para indicar quién es el dueño y el grupo de un archivo. Se puede especificar el usuario (en este caso usuariox) y el grupo (grupoy), separados por el carácter.

chown usuariox:grupoy archivo.txt

chown -R www-data:www-data /var/www/html

Esto se puede observar al realizar el comando ls -l sobre un archivo

ls -l archivo.txt
-rw-r--r-- 1 usuario grupo 1234 Feb 1 12:34 archivo.txt

3.2 Permisos dentro de linux

Ejercicio 1

Crea un archivo llamado archivo1.txt y deja los permisos de forma que:

1. El propietario pueda leer y escribir
2. El grupo solo pueda leer
3. Otros no tengan ningún permiso

Ejercicio 2

Crea un directorio llamado proyecto y configura los permisos para que:

1. El propietario tenga todos los permisos
2. El grupo pueda leer y entrar al directorio
3. Otros no tengan acceso

3.2 Permisos dentro de linux

Ejercicio 1

touch archivo1.txt
chmod 640 archivo1.txt

Ejercicio 2

mkdir proyecto
chmod 750 proyecto

3.2 Permisos dentro de linux

Ejercicio 1

Cambia recursivamente el propietario y grupo del directorio proyecto a:

Usuario: admin
Grupo: desarrolladores

3.2 Permisos dentro de linux

Ejercicio 1

1. **sudo useradd** -m -g desarrolladores admin
   a. -m : crea el directorio personal /home/admin
   b. -g desarrolladores : asigna ese grupo como grupo principal

2. **sudo groupadd** desarrolladores

3. **chown -R** admin:desarrolladores proyecto

3.3 Primeros pasos - Ejemplo

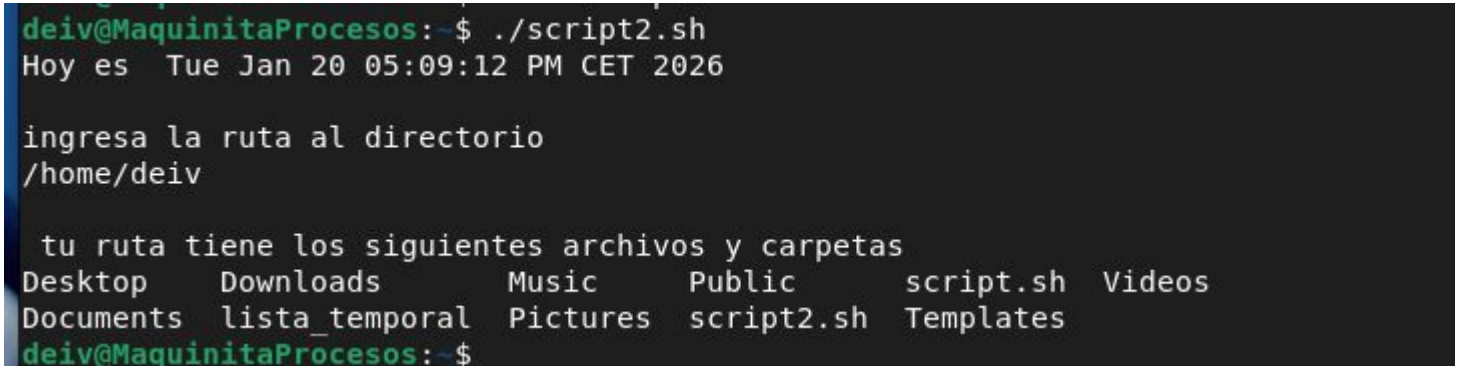
Prueba a correr el siguiente comando

```
#!/bin/bash
echo "Hoy es " `date`

echo -e "\ningresa la ruta al directorio"
read the_path

echo -e "\n tu ruta tiene los siguientes archivos y carpetas: "
ls $the_path
```

3.3 Primeros pasos - Ejemplo



3.4 Tipos de datos - Listas o tuplas

La información puede ser procesada de manera eficiente en estructuras de datos. Las estructuras de datos son formas de organizar los datos. Pueden ser tan complejas como sea necesario. Varios algoritmos requieren el diseño de estructuras de datos con particularidades para facilitar el procesamiento de datos. La primera estructura de datos se deriva del álgebra lineal, específicamente de las tuplas. Una n-tupla es una lista finita con n elementos ordenados. Abajo tenemos una tupla llamada x. El conteo empieza desde 0, hasta n-1.

$$\mathbf{x} = (x_0, x_1, x_2, \ldots, x_{n-1})$$

https://rsg-ecuador.github.io/HerrCompBioinfo/contenido/09_Bash/2_condicionales_arreglos.html

3.4 Tipos de datos - Listas o tuplas

```
$ bichos=(mosca maroposa araña cucaracha saltamontes abeja piojo libelula escarabajo mantis)
$ echo ${bichos[0]}
$ echo ${bichos[1]}
$ echo ${bichos[2]}
$ echo ${bichos[*]}
```

3.4 Tipos de datos - Listas o tuplas

Cambio el valor dentro de una lista
$ echo ${bichos[*]}
$ bichos[4]=hormiga
$ echo ${bichos[*]}
Declaramos un subconjunto dentro de nuestra lista
$ echo ${bichos[*]:5:3}

3.4 Tipos de datos - Listas o tuplas

Y podemos usar el operador += para concatenar dos listas. Concatenamos el anterior con una lista creada con (lombriz escorpion pulga)

```
$ bichos+=(lombriz escorpion pulga)
$ echo ${bichos[*]}
```

3.5 Nomenclatura

En programación de Bash, las convenciones de nomenclatura de variable son los siguientes:

1. Los nombres de variable deberían comenzar con una letra o un guión bajo (_).
2. Los nombres de variable pueden contener letras, números, y guiones bajos (_).
3. Los nombres de variable se distinguen entre mayúsculas y minúsculas.
4. Los nombres de variable no deberían contener espacios o caracteres especiales.
5. Usa nombres descriptivos que reflejen el propósito de la variable.
6. Evitar usar palabras claves reservadas, tales como if, then, else, fi, y así sucesivamente como nombres de variable..
7. Cuidado al poner ` (comillas invertidas) ya que lanzan el comando.

3.5 Nomenclatura

Aquí hay algunos ejemplos de nombres de variable **válidos** en Bash:

1. nombre
2. cuenta
3. _var
4. miVar
5. MI_VAR

Y aquí hay algunos ejemplos de nombres de variable **inválidos**:

1. 2ndvar (el nombre de variable comienza con un número)
2. mi var (el nombre de variable contiene un espacio)
3. mi-var (el nombre de variable contiene un guión)

3.5.1 Ejercicio script

Realiza un script que haga:

1. Muestre un mensaje de bienvenida.
2. Pida al usuario que introduzca su nombre.
3. Guarde el nombre en una variable.
4. Muestre un saludo personalizado.
5. Solicite al usuario que introduzca una ruta a un archivo.
6. Muestre en pantalla el contenido del archivo indicado.

3.5.1 Ejercicio script

```
1. #!/bin/bash
2. echo "Bienvenido al script"
3. echo "Introduce tu nombre:"
4. read nombre
5. echo "Hola, $nombre"
6. echo "Introduce la ruta de un archivo:"
7. read archivo
8. echo "El contenido del archivo es:"
9. cat "$archivo"
```

3.6 Declaraciones condicionales

Expresiones que producen un resultado booleano, sea verdadero o falso, son llamados condiciones. Hay varias formas de evaluar condiciones, incluyendo i**f, if-else, if-elif-else.**

```
if [[ condicion ]];
then
    declaracion
elif [[ condicion ]]; then
    declaracion
else
    haz esto por defecto
fi
```

3.6 Declaraciones condicionales

Podemos usar operadores lógicos tales como AND -a y OR -o para hacer comparaciones que tienen más significado.

if [$a -gt 60 -a $b -lt 100]

3.6.1 Operadores condición

Logical Flag	Meaning	Usage
-gt	Greater Than	`[[ $planets -gt 8 ]]`
-ge	Greater Than or Equal To	`[[ $votes -ge 270 ]]`
-eq	Equal	`[[ $fingers -eq 10 ]]`
-ne	Not Equal	`[[ $pages -ne 0 ]]`
-le	Less Than or Equal To	`[[ $candles -le 9 ]]`
-lt	Less Than	`[[ $wives -lt 2 ]]`
-e	A File Exists	`[[ -e $taxes_2016 ]]`
-d	A Directory Exists	`[[ -d $photos ]]`
-z	Length of String is Zero	`[[ -z $name ]]`
-n	Length of String is Non-Zero	`[[ -n $name ]]`

Logical Operator	Meaning	Usage
=~	Matches Regular Expression	`[[ $consonants =~ [aeiou] ]]`
=	String Equal To	`[[ $password = "pegasus" ]]`
!=	String Not Equal To	`[[ $fruit != "banana" ]]`
!	Not	`[[ ! "apple" =~ ^b ]]`

3.6.2 Ejemplo Condicionales

```
#!/bin/bash

echo "Por favor ingresa un numero: "
read num

if [ $num -gt 0 ]; then
  echo "$num es positivo"
elif [ $num -lt 0 ]; then
  echo "$num es negativo"
else
  echo "$num es cero"
fi
```

3.6.3 Ejercicio condicionales

1. **Script para comprobar si un número es par o impar**

3.6.3 Ejercicio condicionales

Script para comprobar si un número es par o impar

```
1.  #!/bin/bash
2.
3.  echo "Por favor ingresa un número:"
4.  read num
5.
6.  if [ $((num % 2)) -eq 0 ]; then
7.    echo "$num es un número par"
8.  else
9.    echo "$num es un número impar"
10. fi
```

3.6.4 Ejercicio condicionales 2

Crea un script en Bash que realice las siguientes acciones:

1. Muestre el directorio actual en el que se está ejecutando el script.
2. Pida al usuario que introduzca una ruta (archivo o directorio).
3. Compruebe:
   a. Si la ruta es un directorio, mostrar su contenido usando ls.
   b. Si la ruta es un archivo, mostrar su tamaño en bytes usando stat.
4. Finalmente, mostrar los procesos en ejecución utilizando top.

3.6.4 Ejercicio condicionales 2

1. **Crea un script para jugar a piedra, papel o tijeras**



https://rsg-ecuador.github.io/HerrCompBioinfo/contenido/09_Bash/3_bucles.html

3.7 Declaraciones condicionales - Case

```
case expression in
    pattern1)
        # código a ejecutar si la expresión coincide con pattern1
        ;;
    pattern2)
        # código a ejecutar si la expresión coincide con pattern2
        ;;
    pattern3)
        # código a ejecutar si la expresión coincide con pattern3
        ;;
    *)
            # código a ejecutar si ninguno de los patrones de arriba coincide con la expresión
        ;;
esac
```

3.7 Declaraciones condicionales - Case

```
fruta="manzana"
case $fruta in
    "manzana")
        echo "Este es una fruta roja."
        ;;
    "banana")
        echo "Este es una fruta amarilla."
        ;;
    "orange")
        echo "Este es una fruta naranja."
        ;;
    *)
        echo "Fruta desconocida."
        ;;
esac
```

3.8 Bucles While

Bucle While

1. Los bucles while verifican una condición y hacen un bucle hasta que la condición sea true. Necesitamos proveer un contador que vaya incrementándose para controlar la ejecución del bucle.

```
#!/bin/bash
i=1
while [[ $i -le 10 ]] ; do
        echo "$i"
        (( i += 1 ))
done
```

2. En el ejemplo de abajo, ((i += 1)) está la declaración contador que incrementa el valor de i. El bucle lo ejecutará exactamente 10 veces.

3.8 Bucles While

Realiza un bucle que realice las siguientes operaciones:

1. Pida al usuario que introduzca una palabra.
2. Pregunte cuántas veces quiere que se muestre esa palabra.
3. Utilice un bucle while para mostrar la palabra el número de veces indicado, junto con el número de repetición.

3.8 Bucles While

```
1.  #!/bin/bash
2.  echo "Introduce una palabra:"
3.  read palabra
4.  echo "¿Cuántas veces quieres mostrarla?"
5.  read veces
6.  contador=1
7.  while [ $contador -le $veces ]; do
8.    echo "$contador: $palabra"
9.    contador=$((contador + 1))
10. done
```

3.8.2 Bucles For

El bucle for, así como el bucle while, te permite ejecutar declaraciones en un número específico de veces. Cada bucle difiere en su sintaxis y uso.

En el ejemplo, el bucle iterará 5 veces.

```
1. #!/bin/bash
2. for i in {1..5}
3. do
4.     echo $i
5. done
```

3.8.2 Bucles For

```
1. for ciudad in Manila Bangkok Yakarta Kuala Lumpur
2. do
3.   if [[ "$ciudad" == 'Yakarta' ]]; then
4.     break
5.   fi
6.   echo "ciudad: $ciudad"
7. done
8.
9. echo 'Sí, ¡eso es todo!'
```

3.9 Gestión de entrada/salida

Potencial de redirigir entrada y salida. Es una buena manera de utilizar determinadas herramientas sin necesidad de utilizar archivos de almacenamiento intermedio.
Cualquier comando que se ejecuta puede generar dos tipos de salida,

Un resultado. Es decir, aquello para lo que está pensado la herramienta en cuestión. En el caso de ls, que he comentado anteriormente, el resultado es el listado de archivos.
Un estado. El estado se refiere a si la ejecución ha sido correcta. En caso de que la ejecución no haya sido correcta devolverá un mensaje de error para informar al usuario de los detalles de la ejecución.

https://atareao.es/tutorial/terminal/redirigir-entrada-y-salida-en-linux/

3.9 Gestión de entrada/salida

En Linux existen tres tipos de archivos por defecto,

1. stdin ó 0. Está conectado con el teclado.
2. stdout ó 1. Este archivo está conectado con la pantalla.
3. stderr ó 2. Cuando se produce un error, este se envía a este archivo, que también está conectado con la pantalla.

https://atareao.es/tutorial/terminal/redirigir-entrada-y-salida-en-linux/

3.9 Gestión de entrada/salida

Redirigir salida

Para esto tienes dos operadores de redirección, > y <. Así, por ejemplo, para guardar el resultado de ls en un archivo, tan solo tienes que ejecutar el siguiente comando.

Por ejemplo:

ls > salida.txt
ls 1>salida.txt.

https://atareao.es/tutorial/terminal/redirigir-entrada-y-salida-en-linux/

3.9 Gestión de entrada/salida

Añadir contenido a la salida / Realizar un append a un archivo

En este caso se utiliza el operador >>.
Así, por ejemplo cada vez que ejecutes

ls >> salida.txt

El resultado se añadirá al archivo existente.

Para redirigir la salida de error de un comando, es necesario especificar el número del archivo de error además del tipo de archivo 2

ls 2>error.log

https://atareao.es/tutorial/terminal/redirigir-entrada-y-salida-en-linux/

3.9 Gestión de entrada/salida

Redirigir salida y errores

Para esto, es necesario utilizar los números que definen cada una de las salidas estándar. Para esto, tienes dos opciones:

ls >salida.txt 2>&1

Con salida.txt, indica que el resultado va al archivo salida.txt, como has visto anteriormente. Y con 2>&1 indicas que los errores se redirigen a la salida estándar, a 1. Como 1 está redirigido a su vez a salida.txt, ya lo tienes.

ls &>salida.txt

Esta segunda forma es mucho mas simple aunque menos intuitiva que la primera. Aquí también es posible añadir contenido, en lugar de reemplazar. Para ello ls &>>salida.txt

https://atareao.es/tutorial/terminal/redirigir-entrada-y-salida-en-linux/

3.9 Gestión de entrada/salida

Redirigir la entrada estándar

Para utilizar un archivo como entrada, en lugar de la entrada estándar, tienes que utilizar el operador <.

cat < salida.txt

Redirigir entrada y salida estándar

rev < hola.txt > hola_al_reves.txt

El comando rev le da la vuelta a la entrada que ponemos, que redireccionándolo , nos dará un archivo al revés

https://atareao.es/tutorial/terminal/redirigir-entrada-y-salida-en-linux/

3.9 Gestión de entrada/salida

Tuberías

Linux nos da la posibilidad de encadenar comandos. De esta forma la salida de un comando resulta ser la entrada del siguiente. Esto es muy interesante, en tanto en cuanto, obviarás tener que guardar resultados intermedios en archivos.

ls | rev > texto_al_reves.txt

https://atareao.es/tutorial/terminal/redirigir-entrada-y-salida-en-linux/

3.9 Gestión de entrada/salida

Herramientas adicionales

1. **head.** Extrae las primeras líneas de un archivo. Por ejemplo, head -1 sacará la primera línea del archivo`.
2. **tail.** Saca las últimas líneas de un archivo. Por ejemplo, tail -1 te sacará la última línea.
3. **cut.** Imprime parte de una línea de un archivo a una salida estándar. Por ejemplo, cut -d" " -f3 extraerá el campo 3 de la línea, donde los campos están separados por espacios.

https://atareao.es/tutorial/terminal/redirigir-entrada-y-salida-en-linux/

3.9 Gestión de entrada/salida

Ejemplo práctico: Dada esta entrada de ls -lah

```
total 32K
drwxrwxr-x  2 lorenzo lorenzo 4,0K mar 11 09:17 .
drwx------ 42 lorenzo lorenzo 4,0K dic  9 07:47 ..
-rw-rw-r--  1 lorenzo lorenzo 150,0K mar 11 08:56 10.txt
```

Se puede obtener el peso del último archivo (10.txt) de la siguiente manera

```
ls -lah | tail -1 | cut -d" " -f6
```

https://atareao.es/tutorial/terminal/redirigir-entrada-y-salida-en-linux/

3.9.1 Ejercicio ficheros

Ejercicio 1: Recorrer fichero para que saque linea a línea su contenido:

Ejercicios de Bash Shell Script – educatica!

3.10 Ejercicio Bucles

Crea un script en Bash que recorra todos los archivos con extensión .txt del directorio actual y, para cada uno:

1. Muestre el nombre del archivo.
2. Muestre cuántas líneas contiene.

Pista: wc -l < "$archivo" wc muestra la líneas,palabras o bytes de la entrada. Con la opción -l cuenta las líneas del archivo.

3.10 Ejercicio Bucles

```
1.  #!/bin/bash
2.  hay_txt=false
3.  for archivo in *.txt
4.  do
5.      if [ "$archivo" = "*.txt" ]; then
6.          echo "No hay archivos .txt en el directorio."
7.          break
8.      fi
9.
10.     lineas=$(wc -l < "$archivo")
11.     echo "Archivo: $archivo - Líneas: $lineas"
12.     hay_txt=true
13. done
```

3.11 Ejercicio Case

Crea un script llamado doSomething.sh que reciba por parámetro una operación y un valor. Es decir, el script debe recibir dos parámetros siempre.
Si no recibe dos parámetros, el script deberá mostrar un mensaje de error y terminar la ejecución.
El primero parámetro determina la operación a realizar que será una de las siguientes:

- ➔ a. Mostrar la fecha y hora actual.
- ➔ b. Mostrar los últimos 10 usuarios creados en le sistema
- ➔ c. Mostrar los últimos 10 grupos creados en el sistema
- ➔ d. Mandar cuatro paquetes de ping a la dirección ip 8.8.8.8
- ➔ e. Mandar cuatro paquetes de ping al host que se pasa en el segundo parámetro
- ➔ f. Mostrar en pantalla un saludo.

https://www.educatica.es/informatica/sistemas-operativos/gnu-linux-bash/shell-scripts/cases/

3.11 Ejercicio Case

- **Pista:** Recuerda utilizar la función tail para coger las últimas líneas y concatena la salida. Te recomiendo utilizar cut si quieres procesar la entrada
- Recuerda que los campos se pueden utilizar cut

https://www.educatica.es/informatica/sistemas-operativos/gnu-linux-bash/shell-scripts/cases/

3.11 Ejercicio Case

```
#!/bin/bash
opcion=$1
auxiliar=$2
if [ $# -lt 2 ]; then
    echo "ERROR - Se deben pasar dos parámetros"
    exit 1
fi
case $opcion in
    a)
        echo "Fecha y hora"
        date
        ;;
    b)
        echo "Últimos 10 usuarios creados en el sistema"
        tail -10 /etc/passwd | cut -d: -f1
        ;;
    c)
        echo "Últimos 10 grupos creados en el sistema"
        tail -10 /etc/group | cut -d: -f1
        ;;
    d)
        ping -c4 8.8.8.8
        ;;
    e)
        ping -c4 $auxiliar
        ;;
    f)
        echo Hola que tal estas
        ;;
    *)
        echo "No conocemos la operación $opcion"
        ;;
esac
```

https://www.educatica.es/informatica/sistemas-operativos/gnu-linux-bash/shell-scripts/cases/

3.12 Funciones en Bash

Hay dos sintaxis diferentes para declarar funciones bash. La siguiente sintaxis es la más utilizada para crear funciones bash:

```
nombre_de funcion () {
        comandos
          }
```

Por ejemplo:

```
#!/bin/bash

hola () {
echo "Hola Mundo"
}

hola
hola
hola
```

Usando Funciones en Bash

3.12 Funciones en Bash

También podemos poner parámetros dentro de las funciones como cuando se llama a Bash. Por ejemplo:

```
#!/bin/bash
espar () {
if [ $(($1 % 2)) -eq 0 ]; then
echo "$1 es par."
else
echo "$1 es impar."
fi
}

espar 3
espar 4
espar 20
espar 111
```

Usando Funciones en Bash

4. Depuración de código

Es el proceso sistemático de identificar, localizar y corregir errores ("bugs") en un programa para asegurar su correcto funcionamiento.

Establecer la opción set -x

Una de las técnicas más útiles para depurar scripts de Bash es establecer la opción set -x al principio del script. Esta opción permite el modo en depuración, el cual hace que Bash imprima cada comando que ejecuta a la terminal, precedido por un signo +, Esto puede ser increíblemente útil al identificar dónde ocurren los errores en tu script.

4. Depuración de código

```
#!/bin/bash
set -x
# Tu script va aquí
```

4. Depuración de código

Verifica el código de salida

Cuando Bash encuentra un error, pone un código de salida que indica la naturaleza del error. Puedes verificar el código de salida del comando más reciente usando la variable $?. Un valor de 0 indica éxito, mientras que cualquier otro valor indica un error.

4. Depuración de código

```
#!/bin/bash

# Tu script va aquí

if [ $? -ne 0 ]; then
    echo "Hubo un error."
fi
```

4. Depuración de código

Usa declaraciones echo

Otra técnica útil para depurar scripts de Bash es insertar declaraciones echo a lo largo de tú código. Esto puede ayudarte en identificar cuándo ocurren los errores y qué valores están siendo pasados a las variables.

```
#!/bin/bash
# Tu script va aquí
echo "El valor de la variable x es: $x"
# Más código va aquí
```

4. Depuración de código

Resolviendo problemas de crons verificando los logs
Podemos solucionar problemas de crons usando los archivos de log. Los logs son mantenidos para todos los trabajos programados. Puedes verificar en los logs si un trabajo específico se ejecutó como se pretendía o no.

Para Ubuntu/Debian, puedes encontrar los logs de cron en:

/var/log/syslog

Gracias

